Análisis del Diseño RTL y Resultados del Algoritmo GCD mediante FSMD

Facultad de Estudios Superiores Cuautitlán – UNAM

1. RTL

- **Comparator:** Compara los valores de entrada y genera las señales que indican si los números son iguales o si uno es menor que el otro.
- **Subtractor:** Realiza las operaciones de resta entre los dos números. Se utilizan dos unidades para calcular $x - y$ y $y - x$.
- **Multiplexer:** Selecciona los valores que se enviarán a los registros, ya sea las entradas iniciales o los resultados de las restas.
- **Register:** Almacena los valores de los números y permite actualizarlos durante el proceso. También se utiliza un registro para guardar el resultado final.
- **Control Unit:** Controla el funcionamiento del circuito mediante una máquina de estados. Indica cuándo cargar los datos, realizar las comparaciones, seleccionar las restas y finalizar el proceso.
- **GCD FSMD:** Es el módulo principal, donde se conectan los módulos anteriores para que trabajen en conjunto y realicen el cálculo del máximo común divisor.

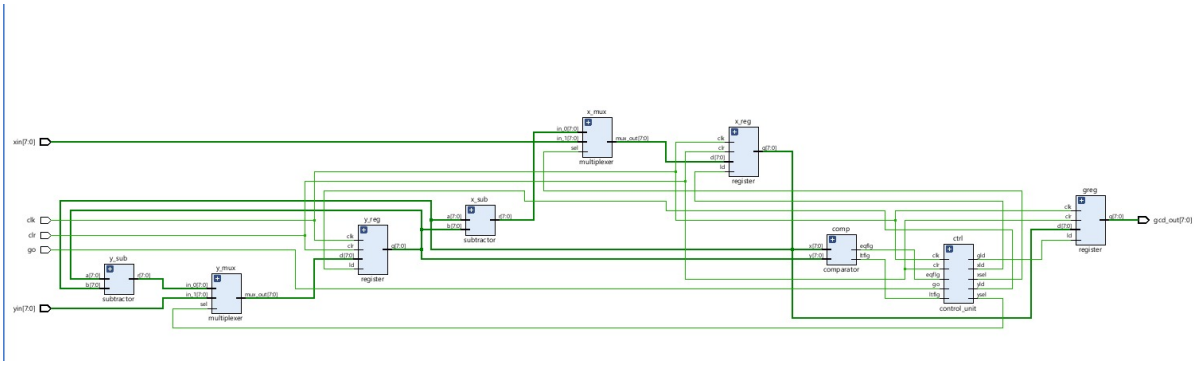


Figura 1: Diagrama RTL

2. Resultados de la simulación

En cada una utilicé diferentes valores de entrada y revisé que el resultado mostrado en la salida `gcd_out` fuera el máximo común divisor de los números ingresados.

Caso 1: GCD (70, 15)

Para la primera prueba ingresé los valores 70 y 15 en decimal, que corresponden a `46` y `0f` en hexadecimal. Durante la simulación, el circuito realizó las comparaciones y restas necesarias hasta que los dos valores llegaron a ser iguales. Al finalizar, la salida `gcd_out` mostró `05` en hexadecimal, que corresponde a 5 en decimal.

Caso 2: GCD (48, 18)

En la segunda prueba utilicé los valores 48 y 18 en decimal, representados como `30` y `12` en hexadecimal. El circuito realizó las comparaciones correspondientes y fue actualizando los valores mediante las restas sucesivas. Finalmente, la salida `gcd_out` mostró `06` en hexadecimal, equivalente a 6 en decimal.

Caso 3: GCD (25, 25)

Para la última prueba ingresé dos valores iguales de 25 en decimal, representados como `19` en hexadecimal. En este caso, el comparador detectó que ambos números eran iguales mediante la señal `eqflg`, por lo que no fue necesario realizar ninguna resta. La máquina de estados pasó al estado `done` y la salida `gcd_out` mostró `19` en hexadecimal, que corresponde a 25 en decimal.

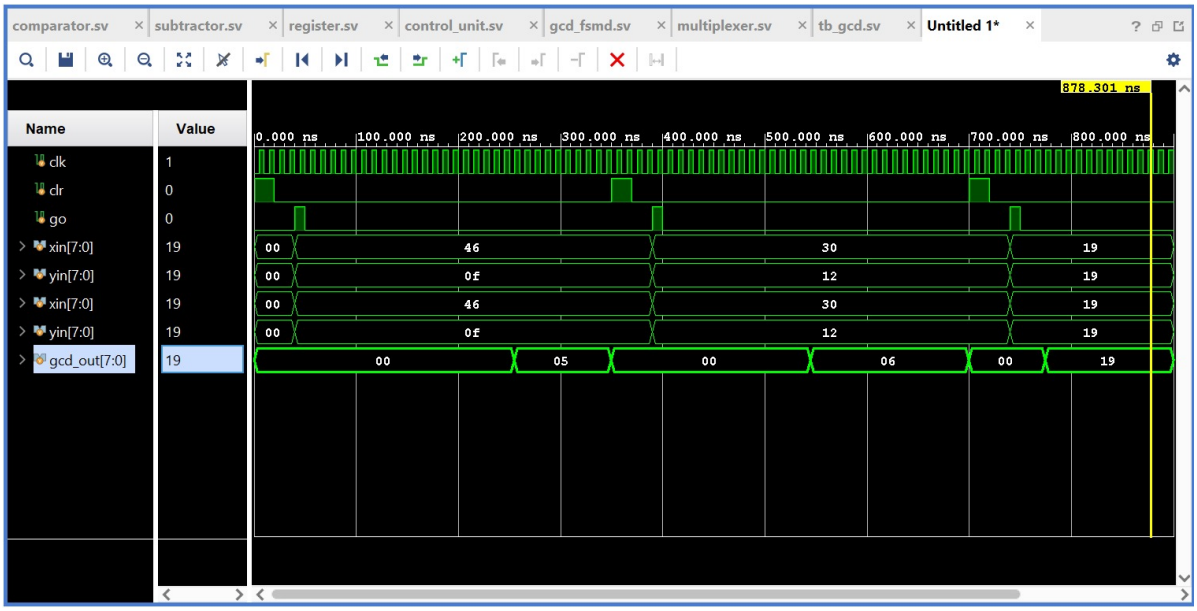


Figura 2: Simulaci'on

3. Análisis de los resultados

Al revisar los resultados de las tres pruebas, pude comprobar que el circuito realizó correctamente el cálculo del máximo común divisor para los valores de entrada utilizados.

En los dos primeros casos, observé que el circuito realizó las comparaciones y restas sucesivas hasta que los valores de los registros fueron iguales. Esto permitió comprobar que la unidad de control selecciona la operación que corresponde dependiendo de cuál de los dos números sea mayor.

En el tercer caso, el funcionamiento fue diferente, ya que los dos números ingresados eran iguales desde el inicio. El circuito detectó esta condición mediante la señal `eqflg` y pasó al estado `done` sin necesidad de realizar restas.

Los resultados obtenidos fueron los esperados: 5 para el primer caso, 6 para el segundo y 25 para el tercero. Con estas pruebas pude comprobar que la máquina de estados y la ruta de datos trabajan de manera coordinada para realizar el algoritmo de Euclides mediante restas sucesivas.

Por lo tanto, los resultados de la simulación permiten verificar que el diseño implementado cumple con el objetivo de calcular el máximo común divisor de dos números de 8 bits.